Mini E-Commerce Backend

Admin Module Blueprint

1. Admin Module Overview

Responsibilities:
• Add New Product
• Update Existing Product
• Delete Product
• Manage Product Catalog
• Manage Order Status
• Protect Admin Routes

2. Admin APIs

Create Product

Item	Value
Method	POST
Endpoint	/products
Authorization	Admin
Response	201 Created

Update Product

Item	Value
Method	PATCH
Endpoint	/products/:id
Authorization	Admin
Response	200 OK

Delete Product

Item	Value
Method	DELETE
Endpoint	/products/:id
Authorization	Admin
Response	200 OK

Update Order Status

Item	Value

Method	PATCH
Endpoint	/orders/:id
Authorization	Admin
Response	200 OK

3. Admin Request Flow

```
Client → JWT Middleware → Admin Middleware → Controller → MongoDB → Response
```

4. Admin Middleware Flow

```
Request → Authorization Header → JWT Verify → req.user → Check Role → role == 'admin' → YES → next() / NO → 403 Forbidden
```

5. Authorization Matrix

Operation	User	Admin
Browse Products	Yes	Yes
Get Product	Yes	Yes
Add Product	No	Yes
Update Product	No	Yes
Delete Product	No	Yes
Update Order Status	No	Yes

6. Admin Responsibilities

- Add new products.
- Update product information.
- Delete products.
- Manage product inventory.
- Update order status.

7. Folder Structure

```
middlewares/
    admin.middleware.js

controllers/
    product.controller.js
    order.controller.js

routes/
    product.route.js
    order.route.js
```

8. HTTP Status Codes

Code	Description
200	Success
201	Resource Created
401	Unauthorized
403	Forbidden
404	Resource Not Found
500	Internal Server Error

9. Common Error Scenarios

Error	Status Code
Missing Token	401
Invalid Token	401
User is not Admin	403
Product Not Found	404
Order Not Found	404

10. Security Rules

• Every Admin Route must pass through JWT Middleware.
• Every Admin Route must pass through Admin Middleware.
• Normal users cannot access administrative operations.
• Only authenticated admins can manage products and orders.

11. Testing Checklist

• Add Product
• Update Product
• Delete Product
• Update Order Status
• JWT Authentication
• Admin Authorization
• Invalid Token
• User Role Validation

12. Interview Notes

Why use Admin Middleware?
To separate authorization logic from controllers and make the code reusable.

Why check role after JWT?
We must identify the user before checking admin privileges.

Authentication vs Authorization?
• Authentication verifies who the user is.
• Authorization determines what the user can do.

Why return 403 instead of 401?
The user is authenticated but lacks permission.

Why restrict product management APIs?
To protect the product catalog from unauthorized modifications.

13. Sequence Diagram

`Client → Request → JWT Middleware → Admin Middleware → Controller → Database → Response`

14. Best Practices

• Keep authorization logic inside middleware.
• Reuse Admin Middleware for all protected admin routes.
• Return meaningful HTTP status codes.
• Never expose sensitive user information in responses.
• Validate all incoming requests before processing.